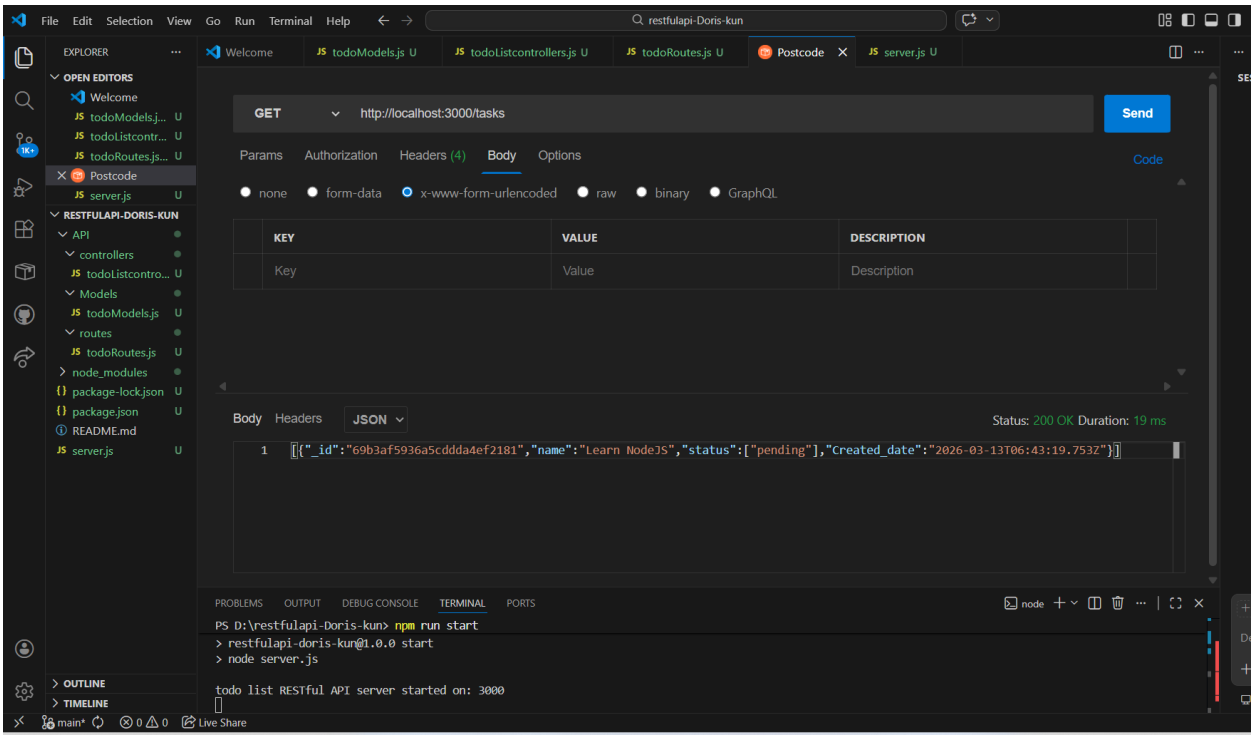
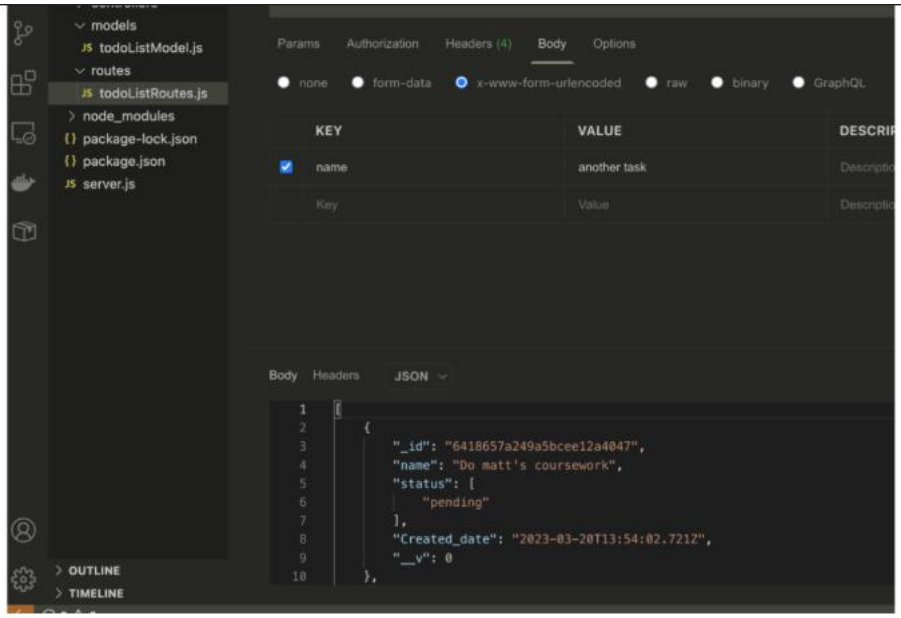
Name: Phạm Thái Nam
MSSV: GCS220749





4) Your data in your mongoDB, either the University one or your localhost one

```
var TaskSchema = new Schema({
  name: {
    type: String,
    required: 'Kindly enter the name of the task'
  },
  Created_date: {
    type: Date,
    default: Date.now
  },
  status: {
    type: [{
      type: String,
      enum: ['pending', 'ongoing', 'completed']
    }],
    default: ['pending']
  }
});

module.exports = mongoose.model('Tasks', TaskSchema);
```

```
PS D:\restfulapi-Doris-kun> npm run start
> restfulapi-doris-kun@1.0.0 start
> node server.js

todo list RESTful API server started on: 3000
```


```
module.exports = mongoose.model('Tasks', TaskSchema);
```

```
PS D:\restfulapi-Doris-kun> npm run start
> restfulapi-doris-kun@1.0.0 start
> node server.js

todo list RESTful API server started on: 3000
```


GET http://localhost:3000/tasks Send

Params Authorization Headers (4) Body Options Code

none form-data x-www-form-urlencoded raw binary GraphQL

KEY	VALUE	DESCRIPTION
name	anothertask	Description
Key	Value	Description

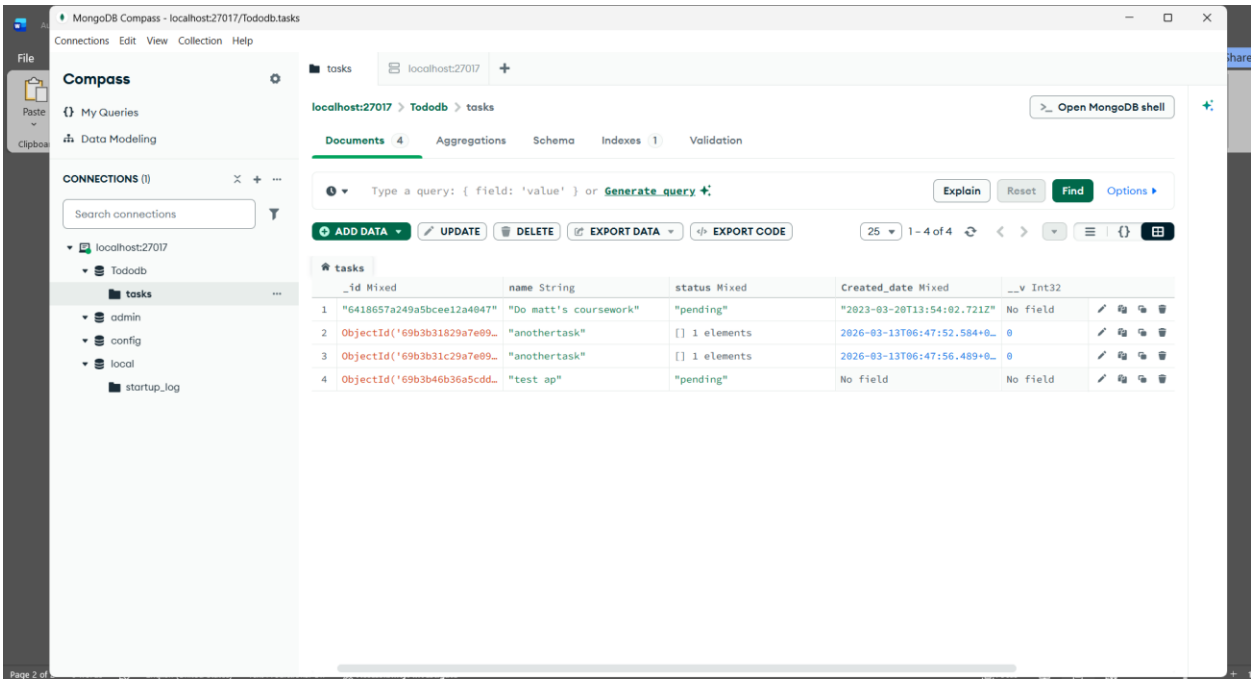
Body Headers JSON Status: 200 OK Duration: 8 ms


```
[{"_id":"6418657a249a5bcee12a4047","name":"Do matt's coursework","status":["pending"],"Created_date":"2023-03-20T13:54:02.721Z"},
```



```
PS D:\restfulapi-Doris-kun> npm run start
> restfulapi-doris-kun@1.0.0 start
> node server.js

todo list RESTful API server started on: 3000
```



This lab task demonstrates how to build a simple RESTful API for managing a to-do list using Node.js, Express, and MongoDB with Mongoose. The project is structured into several main parts: models, controllers, routes, and the server configuration. This structure helps organize the code and follows common backend development practices.

First, the **TaskSchema** is defined using Mongoose in the model file. The schema describes the structure of the task document stored in MongoDB. Each task contains a **name**, which is a required string representing the task title. If the name is not provided, the system will return an error message asking the user to enter the task name. The schema also includes a **Created_date**, which automatically records the time when the task is created using Date.now. Another important field is **status**, which represents the state of the task. The status can only take specific values such as *pending*, *ongoing*, or *completed*, ensuring data consistency.

After defining the schema, the model is exported and used by the controllers to perform operations such as creating or retrieving tasks. The Express server runs on port 3000, allowing API requests to be sent through endpoints like /tasks.

To test the API, Postcode is used to send HTTP requests. In the example shown, a request is sent to retrieve tasks from the server. The response returns task data in JSON format, including the task name, status, creation date, and a unique ID generated by MongoDB.

Finally, MongoDB Compass is used to visually confirm that the task data has been successfully stored in the database. This demonstrates how the API interacts with the database and verifies that the backend system is functioning correctly.